

Deploy self-hosted — inventory-its

Internacional Travessias — guia de instalação e operação

Deploy self-hosted (dentro da rede da Internacional Travessias)

0. Antes de começar
 1. Instalar o Docker
 2. Clonar o projeto no servidor
 3. Configurar o `.env`
 4. Subir os containers
 5. Migrando os dados que já existem no Neon
 6. Liberar as portas no firewall (pra rede local)
 7. Apontar o app mobile pro servidor novo
 8. Backup do banco
 9. Atualizações futuras
- Troubleshooting rápido

Deploy self-hosted (dentro da rede da Internacional Travessias)

Este guia tira o sistema do Neon (banco) + Render (backend) + onde quer que o painel estivesse hospedado, e coloca tudo rodando num servidor da empresa, acessível só pela rede interna.

Estrutura final: 3 containers Docker no mesmo servidor — `postgres` (banco), `backend` (API) e `web` (painel) — orquestrados pelo `docker-compose.yml` na raiz do repositório. O app mobile continua sendo instalado normalmente nos celulares; ele só passa a apontar pro IP desse servidor em vez do endereço antigo.

O painel web tem uma tela de **Monitoramento** (`/logs` , só pra admin) que mostra em tempo real os eventos de sucesso e erro do sistema inteiro — painel, app mobile e backend. Isso é gravado numa tabela própria (`logs_sistema`), já incluída no `schema.sql` do passo 4 — não precisa de nenhum passo extra de configuração.

0. Antes de começar

- Servidor Linux (Ubuntu/Debian) com **IP fixo** na rede da empresa (ex: `192.168.1.50`) — se for DHCP, reserve o IP no roteador/DHCP server, porque esse endereço fica “gravado” no painel web (ver passo 3) e em cada celular com o app.
- Acesso root/sudo nesse servidor.
- Uns 2 GB de RAM livres e 10 GB de disco já são confortáveis pro tamanho atual do inventário.

1. Instalar o Docker

```
# Remove versões antigas/conflitantes, se existirem
sudo apt-get remove -y docker docker-engine docker.io containerd runc &>/dev/null

sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]"
```

```

https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME" stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin

# Deixe seu usuário rodar docker sem sudo (precisa logar de novo depois)
sudo usermod -aG docker $USER

```

Se o servidor for Debian em vez de Ubuntu, troque `ubuntu` por `debian` nos dois comandos que montam a URL do repositório.

Confirme que funcionou:

```

docker --version
docker compose version

```

2. Clonar o projeto no servidor

```

cd /opt
sudo git clone https://github.com/joaoribeirodev/inventory-its.git
sudo chown -R $USER:$USER inventory-its
cd inventory-its

```

3. Configurar o `.env`

```

cp .env.example .env
nano .env

```

Preencha principalmente: - `POSTGRES_PASSWORD` — uma senha forte, só esse servidor vai saber. - `JWT_SECRET` — uma string aleatória longa (ex: `openssl rand -hex 32`). - `NEXT_PUBLIC_API_URL` — troque `SEU_IP_AQUI` pelo IP fixo do servidor (ex: `http://192.168.1.50:3001`). **Esse é o ponto mais fácil de esquecer** — se errar aqui, o painel sobe mas não consegue falar com a API pra quem acessa de outro PC (vai parecer que “não conecta”, mesmo com os containers rodando). - `SEED_ADMIN_*` — credenciais do primeiro admin que o `npm run seed` vai criar.

4. Subir os containers

```

docker compose up -d --build
docker compose ps      # os 3 devem aparecer como "running"/"healthy"

```

Crie as tabelas no banco novo. **Use o SQL puro, não o `prisma db push`** — esse comando precisa baixar um binário de `binaries.prisma.sh` na primeira vez, e esse domínio específico costuma ficar bloqueado em rede corporativa (confirmado nesse projeto — o `npm` / Docker Hub funcionam normalmente, só esse host do Prisma que não passa). O arquivo `backend/prisma/schema.sql` tem o mesmo schema em SQL — aplicar ele só fala com o Postgres local, nenhuma chamada externa:

```

cat backend/prisma/schema.sql | docker compose exec -T postgres \
psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"

```

(Se as variáveis `$POSTGRES_USER` / `$POSTGRES_DB` não estiverem no seu shell atual, rode `export $(grep -v '^#' .env | xargs)` antes, ou simplesmente troque pelos valores literais que você colocou no `.env`.)

Se a sua rede **não** bloquear o Prisma (vale testar — algumas não bloqueiam), `docker compose exec backend npx prisma db push` faz a mesma coisa direto do `schema.prisma`. Mas se já se sabe que bloqueia, nem tente — vai só travar/dar timeout.

Se você **não** vai migrar dados antigos (começando do zero), crie o primeiro admin:

```
docker compose exec backend npm run seed
```

Teste abrindo `http://IP_DO_SERVIDOR:3000` no navegador de outro PC da rede.

5. Migrando os dados que já existem no Neon

Se o inventário já tem itens/usuários cadastrados no banco antigo (Neon), em vez do seed acima, traga os dados de lá pra dentro do container novo. Isso pode ser feito da sua própria máquina (não precisa ser no servidor), desde que tenha o `psql` / `pg_dump` instalado (ou rode via Docker também, como no comando abaixo).

```
# 1) Dump do banco antigo (pegue a connection string no painel do Neon)
docker run --rm -v "$PWD":/dump postgres:16-alpine \
  pg_dump "postgresql://usuario:senha@ep-exemplo.neon.tech/inventory_its?sslmode=require" \
  --no-owner --no-privileges -f /dump/neon-backup.sql

# 2) Copia o dump pro servidor (se o passo 1 foi feito em outra máquina)
scp neon-backup.sql usuario@IP_DO_SERVIDOR:/opt/inventory-its/

# 3) Já no servidor, com os containers rodando e as tabelas já criadas
# (schema.sql do passo 4), restaura os dados:
cd /opt/inventory-its
cat neon-backup.sql | docker compose exec -T postgres \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"
```

Depois de restaurar, **não** rode o `npm run seed` (criaria um admin duplicado) — os usuários que já existiam no Neon vêm junto no dump, login e senha continuam os mesmos.

Como o campo `usuario` (login por nome de usuário) é recente, depois da restauração rode o backfill pra preencher ele nas contas antigas:

```
docker compose exec backend node scripts/preencher-usuario-login.js --confirmar
```

6. Liberar as portas no firewall (pra rede local)

Se o servidor usa `ufw`:

```
sudo ufw allow from 192.168.1.0/24 to any port 3000 proto tcp
sudo ufw allow from 192.168.1.0/24 to any port 3001 proto tcp
```

Troque `192.168.1.0/24` pela faixa de IP real da rede da empresa. Isso libera as portas só pra quem está na rede interna — nada se abre pra internet.

7. Apontar o app mobile pro servidor novo

Pra quem já tem o app instalado: abrir **Configurações** dentro do app → trocar o endereço do servidor pra `http://IP_DO_SERVIDOR:3001` → “Testar e reconectar”. Isso dispara uma sincronização completa puxando os dados do servidor novo.

Pra instalações futuras do app (novo APK), vale já deixar esse endereço como padrão em `app/app.json`:

```
"extra": {
  "defaultServerUrl": "http://IP_DO_SERVIDOR:3001"
}
```

Só tem efeito em quem instalar o app depois dessa mudança — quem já tem o app instalado precisa trocar manualmente nas Configurações mesmo (ou você faz isso pra cada aparelho, já que são poucos).

8. Backup do banco

Com os dados morando só nesse servidor agora, backup passa a ser responsabilidade de vocês (antes o Neon cuidava disso). O jeito mais simples é um cron diário rodando `pg_dump` :

```
# /opt/inventory-its/backup.sh
#!/bin/bash
cd /opt/inventory-its
mkdir -p backups
docker compose exec -T postgres pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \
| gzip > "backups/inventory-its-$(date +%Y%m%d-%H%M%S).sql.gz"
# mantém só os últimos 14 backups
ls -t backups/*.sql.gz | tail -n +15 | xargs -r rm
```

```
chmod +x backup.sh
crontab -e
# adiciona a linha (todo dia às 2h da manhã):
0 2 * * * /opt/inventory-its/backup.sh
```

Idealmente copie os `.sql.gz` pra outro lugar também (um NAS, outro servidor) — backup que só existe na mesma máquina do banco não protege contra a máquina falhar.

9. Atualizações futuras

Quando eu (ou você) mudar algo no código e enviar pro GitHub:

```
cd /opt/inventory-its
git pull
docker compose up -d --build
```

Se a mudança alterou o `schema.prisma` (tabela/campo novo), **não** roda `prisma db push` (mesmo bloqueio de rede). Nesses casos eu sempre vou te passar junto um pequeno `ALTER TABLE ...` pra aplicar do mesmo jeito que o `schema.sql` inicial:

```
echo "ALTER TABLE ..." | docker compose exec -T postgres \
psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"
```

E atualizo o `backend/prisma/schema.sql` pra refletir o estado novo, pra ficar documentado (mesmo sem rodar ele de novo, serve de referência de “como o banco deveria estar”).

Troubleshooting rápido

- `docker compose ps` **mostra o backend reiniciando sem parar** → `docker compose logs backend` — geralmente é `DATABASE_URL` errado ou o Postgres ainda não ficou “healthy” (espere uns segundos).
- **Painel abre mas dá erro ao fazer login / carregar itens** → quase sempre é `NEXT_PUBLIC_API_URL` apontando pro endereço errado no `.env` — lembre que esse valor fica “queimado” dentro do build do painel, então depois de corrigir o `.env` é preciso `docker compose up -d --build web` de novo (só reiniciar não basta).
- **Funciona no navegador do próprio servidor mas não nos outros PCs** → geralmente é firewall (passo 6) ou o `NEXT_PUBLIC_API_URL` apontando pra `localhost / 127.0.0.1` em vez do IP real do servidor.
- `docker compose up -d --build` **falha no próprio backend, com erro mencionando `binaries.prisma.sh` ou “Failed to fetch”** → é o mesmo bloqueio de rede do `prisma db push`, só que acontece durante o build (o `Dockerfile` roda `prisma generate` uma vez, que também baixa um binário de lá — isso é inevitável, o Prisma Client não funciona sem ele, mas só precisa baixar UMA vez, não em toda consulta). Costuma ser bem menos provável de ser bloqueado que o `db push` (filtros corporativos tendem a liberar o que é necessário pra instalar pacotes, e esse download acontece junto do `npm install` /build), mas se acontecer, a saída é buildar a imagem numa máquina com internet livre (seu

notebook, por exemplo) e levar ela pro servidor:

```
# na máquina com internet livre, dentro da pasta do projeto:
docker compose build backend
docker save inventory-its-backend | gzip > inventory-its-backend.tar.gz
scp inventory-its-backend.tar.gz usuario@IP_DO_SERVIDOR:/opt/inventory-its/

# no servidor:
cd /opt/inventory-its
docker load < inventory-its-backend.tar.gz
docker compose up -d
```